

BCS202L & BCS202P- Data Structures and Algorithms

Dr. Priyanka N

Assistant Professor Senior Grade I

School of Computer Science & Engineering
VIT,Vellore.

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	Linear Data Structures	7 hours
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	Searching and Sorting	7 hours
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	Trees	6 hours
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		
Module:5	Graphs	6 hours
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.		
Module:6	Hashing	4 hours
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.		
Module:7	Heaps and AVL Trees	5 hours
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).		

BCS202L- Data Structures and Algorithms

Text Books:

1. Mark A. Weiss, Data Structures & Algorithm Analysis in C++, 4 th Edition, 2013, Pearson Education.

Reference Books:

1. Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft, Data Structures and Algorithms, 1983, Pearson Education.
2. Horowitz, Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C, 2008, 2nd Edition, Universities Press.
3. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Module 3: Sorting Algorithms

- Sorting Problem
 - Bubble Sort
 - Insertion Sort
 - Selection Sort
 - Analysis of Sorting Algorithms

The Sorting Problem

- **Input:**

- A sequence of n numbers $a_1, a_2, \dots, a_n$

- **Output:**

- A permutation (reordering) $a_1', a_2', \dots, a_n'$ of the input sequence such that $a_1' \leq a_2' \leq \dots \leq a_n'$

Some Definitions

- Internal Sort
 - The data to be **sorted** is all stored in the computer's main memory.
- External Sort
 - Some of the data to be sorted might be stored in some external, slower, device.
- In Place Sort
 - The amount of extra space required to sort the data is constant with the input size.

Bubble Sort

- Idea:
 - Repeatedly pass through the array
 - Swaps adjacent elements that are out of order
 - Not suitable for large data sets.
 - Easier to implement, But slower.
- Bubble sort is majorly used where
 - Complexity is not Preferred
 - Simplicity is preferred

Bubble Sort- Working – Iteration I

- Let the elements of array be

13	32	26	35	10
----	----	----	----	----

Sorting will start from the initial two elements. Let compare them to check which is greater.

13	32	26	35	10
----	----	----	----	----

Here, 32 is greater than 13 ($32 > 13$), so it is already sorted. Now, compare 32 with 26.

13	32	26	35	10
----	----	----	----	----

Here, 26 is smaller than 32. So, swapping is required. After swapping new array will look like -

13	26	32	35	10
----	----	----	----	----

Now, compare 32 and 35.

Bubble Sort- Working – Iteration I

13	26	32	35	10
----	----	----	----	----

- Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.
- Now, the comparison will be in between 35 and 10.

13	26	32	35	10
----	----	----	----	----

- Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. **After first iteration, the array will be like below by placing the largest element in its appropriate position.**

13	26	32	10	35
----	----	----	----	----

Now, move to the second iteration.

Bubble Sort- Working – Iteration 2

- Same process will be followed for second iteration.

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

- Here, 10 is smaller than 32. So, swapping is required.
After swapping, the array will be

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

- After Second iteration, the array will be like above by placing the **second largest element** in its appropriate position. Now, move to the third iteration.

Bubble Sort- Working – Iteration 3

- Same process will be followed for third iteration.

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

- **10 is smaller than 26.** So, swapping is required. After swapping, the array will be

13	10	26	32	35
----	----	----	----	----

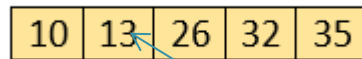
13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

- **After Third iteration, the array will be like above by placing the **third largest element** in its appropriate position & move to the fourth iteration**

Bubble Sort- Working – Iteration 4

- Similarly, after the fourth iteration, the array will be



10	13	26	32	35
----	----	----	----	----

- Hence, there is no swapping required, so the array is completely sorted.
- **After Fourth iteration, the array will be like above by placing the **fourth largest element** in its appropriate position.**

Algorithm Bubble Sort (A [], n)

// A is Array of integers

// N is Number of Elements

```
{
  For i=1 to n do
  {
    For j= 1 to n-i do
    {
      if(a[j]>a[j+1])// Checking Adjacent Elements
      {
        temp=a[j];
        a[j]=a[j+1];
        a[j+1]=temp;
      }
    } // End of j loop
  } // End of i loop
} // End of the Program
```

Bubble Sort - Complexity

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of bubble sort is $O(n)$.
- **Average Case Complexity** - It occurs when the array elements are in random order that is not properly ascending and not properly descending. The average case time complexity of bubble sort is $O(n^2)$.
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of bubble sort is $O(n^2)$.

Insertion Sort - Idea

- Consider the given list of elements as two parts. **Sorted and Unsorted**. By considering the first element only, it is taken as **Sorted**. Remaining other elements are **UnSorted**
- Pick the element from the unsorted part and Compare with all the elements in sorted sub-list.
- Shift all the elements in sorted sub-list that is greater than the value to be sorted.
- Insert the value in the appropriate place.
- Repeat until list is sorted.
- It is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$, where **n** is the number of items.

Working of Insertion Sort

- Consider the following list of elements



- Insertion sort compares the first two elements.



- It finds that both 14 and 33 are already in ascending order. So 14 is taken in **sorted sub-list**.



- Insertion sort moves ahead and compares 33 with 27.



- Finds that 33 is not in the correct position.



- It swaps 33 with 27. **It also checks with all the elements of sorted sub-list.** Here we see that the sorted sub-list has only one element **14**, and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.



- By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10.



- These values are not in a sorted order.



- So we swap them.



- However, swapping makes 27 and 10 unsorted.



- Hence, we swap them too.



- Again we find 14 and 10 in an unsorted order.



- We swap them again. By the end of third iteration, we have a sorted sub-list of 3 items.



This process goes on until all the unsorted values are covered in a sorted sub-list.

Algorithm Insertion sort (A [], n)

```
{
    for(i=1,i<n,i++)//Consider unsorted list as
2 nd Element to n th element in array
    {
        temp=a[i];// Assigning first element in
unsorted part to the variable “item”
        j=i-1;
        while((j>=0)&&(a[j]>temp))

        {
            a[j+1]=a[j];
            j=j-1;
        }
        a[j+1]=temp;
    }// End of j loop
}// End of the program
```

Characteristics of Insertion Algorithm

- It is efficient for smaller data sets, but very inefficient for larger lists.
- Insertion Sort is adaptive, that means it reduces its total number of steps if given a partially sorted list, hence it increases its efficiency.
- Its space complexity is less. Insertion sort requires a single additional memory space.
- **Overall time complexity of Insertion sort is $O(n^2)$.**

Insertion Sort - Complexity

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Insertion Sort - Merits

- Simple and easy
- Efficient for small data
- If the input list is sorted beforehand (partially) then insertions sort takes $O(n+d)$ where d is the number of inversions
- Chosen over bubble sort and selection sort, although all have worst case time complexity as $O(n^2)$
- Maintains relative order of the input data in case of two equal values (stable)
- It requires only a constant amount $O(1)$ of additional memory space (in-place Algorithm)